Belief State Tracking for the ABIDES Daily Investor: Five-Tracker Comparison

Belief-Aware RL Project

May 2026

Abstract

The ABIDES daily-investor environment presents a partially observable Markov decision process: the RL agent observes market prices and order-book quantities but cannot directly see the latent *fundamental value*—the hidden Ornstein-Uhlenbeck (OU) process around which background value agents anchor their bids and asks. We implement and compare five approaches to tracking this hidden state online: (i) a **Kalman filter** exploiting the known OU model structure; (ii) a **bootstrap particle filter** (SIR, $N = 300$) that handles non-Gaussian posteriors and megashocks; (iii) a **Kim regime-switching filter** (Kim 1994) running three parallel Kalman filters over VALUE/MOMENTUM/NOISE regimes; and (iv) a **single-layer LSTM** trained in a self-supervised fashion. All produce a belief deviation $b_t = (\hat{F}_t - \text{mid}_t)/\bar{r}$ used as the sole signal for a threshold trading rule over 20 test episodes. Three of the four belief trackers beat the raw direction-feature baseline (Sharpe −0.68, win rate 20%): Kalman achieves Sharpe +0.50 (win rate 60%, mean P&L +7,670¢); Particle achieves the highest mean P&L +8,703¢ (Sharpe +0.43, win rate 65%); Kim achieves the lowest drawdown (0.021) with Sharpe +0.32; LSTM fails (−48,500¢) due to miscalibration. We analyse each tracker's behaviour and outline next steps for RL integration.

Contents

1 Background and Motivation **3**
- 1.1 The ABIDES Latent State Problem . . . 3
- 1.2 POMDP Framing . . . 3
- 1.3 Sweep Context . . . 3

2 The Kalman Filter Belief Tracker **4**
- 2.1 OU Process Model . . . 4
- 2.2 Parameter Derivation . . . 4
- 2.3 Kalman Predict-Update Equations . . . 5
- 2.4 Steady-State Analysis . . . 5
- 2.5 Interpretation as EMA . . . 5
- 2.6 Threshold Calibration . . . 5
- 2.7 Implementation . . . 6

3 The LSTM Belief Tracker **6**
- 3.1 Motivation . . . 6
- 3.2 Architecture . . . 6
- 3.3 Training Data Collection . . . 7

3.4 Training Objective . . . 7
3.5 Inference Wrapper . . . 7

4 The Particle Filter Belief Tracker **7**
4.1 Motivation . . . 7
4.2 Algorithm . . . 7
4.3 Implementation . . . 8

5 The Kim Regime-Switching Filter **8**
5.1 Motivation . . . 8
5.2 Regimes . . . 8
5.3 Algorithm (Kim 1994 Collapse Approximation) . . . 9
5.4 Regime-Aware Trading Rule . . . 9

6 Experiments **9**
6.1 Setup . . . 9
6.2 Agents . . . 10
6.3 Main Results . . . 11

7 Analysis **11**
7.1 Why the Kalman Filter Works . . . 11
7.2 Why the Particle Filter Nearly Matches Kalman . . . 12
7.3 Why the Kim Filter Is More Conservative . . . 12
7.4 Threshold Calibration and the All-Hold Failure Mode . . . 12
7.5 Why the LSTM Fails . . . 12
7.6 How to Fix the LSTM . . . 13

8 Comparison of All Trackers **14**

9 Next Steps **14**
9.1 Immediate: Fix the LSTM Tracker . . . 14
9.2 RL Integration . . . 14
9.3 Particle Filter with Megashocks Enabled . . . 15
9.4 Generalisation Across Sweep Parameters . . . 15

1 Background and Motivation

1.1 The ABIDES Latent State Problem

The ABIDES `rmsc04` market configuration places background agents of three types into the simulation:

- **Value agents (102 by default)** maintain Bayesian posteriors over the latent OU fundamental F_t and submit limit orders accordingly. Their aggregate order flow anchors the mid-price near F_t.
- **Noise agents (1,000)** submit random one-shot market orders that add microstructure noise to prices and spreads.
- **Momentum agents (12)** use moving-average crossover rules and create short-term autocorrelated pressure.

The RL agent *cannot observe* F_t directly. It sees only the seven-dimensional public observation

$$o_t = [\text{holdings},\ \text{imbalance},\ \text{spread},\ \text{direction},\ \Delta p_{t-1},\ \Delta p_{t-2},\ \Delta p_{t-3}],$$

where direction $= \text{mid}_t - \text{last_transaction}_t$ and Δp_k are lagged mid-price differences over the three preceding 60-second timesteps.

A profitable value-investing strategy requires knowing the sign and magnitude of $F_t - \text{mid}_t$. With only three lags of noisy price data, a fixed MLP policy must implicitly learn to filter F_t from its observation history—a task it performs poorly. This motivates an explicit belief-state module that estimates F_t online and augments o_t with the estimate before passing it to the policy.

1.2 POMDP Framing

Let the hidden state be the current fundamental $F_t \in \mathbb{R}$. The observation o_t is a noisy projection of F_t (mid-price is the most direct projection, with noise from spread, momentum effects, and random orders). A *belief state* is any sufficient statistic of the observation history that captures the posterior $p(F_t \mid o_{1:t})$. For linear-Gaussian dynamics—the OU model—this posterior is Gaussian and is tracked exactly by a Kalman filter. For unknown or nonlinear dynamics, a parametric approximator (LSTM) can learn a surrogate belief state from data.

We implement and compare both.

1.3 Sweep Context

Prior to the belief-tracker experiments, a one-at-a-time (OAT) parameter sweep (`sweep.py`) was run over twelve background parameters including `num_value_agents`, `kappa_oracle`, `fund_vol`, `num_momentum_agents`, and market-maker parameters. The sweep established which parameters most strongly modulate strategy P&L and confirmed that the VALUE strategy (direction-feature threshold) performs best under the default calibration. Key observations from the simulation-level baselines (`run1_data.csv`, `liquidity_shift` variant, 5 seeds) are shown in Table 1.

Agent type	Mean P&L (¢)	Std	Sharpe	Win%
NOISE	+221	35,177	0.006	40.9%
VALUE	+875,621	399,356	2.19	96.5%
MOMENTUM	−5,045,988	1,168,297	−4.32	0.0%
MARKET_MAKER	−2,209,716	2,322,686	−0.95	0.0%

Table 1: Simulation-level baselines from `run1_data.csv`. Value agents operating with their real Bayesian oracle and Kalman filter achieve a Sharpe of 2.19. The gym-level VALUE heuristic (Table 4) uses only the direction feature and performs far worse, motivating a richer belief representation.

The large gap between the *simulation-level* VALUE agent (Sharpe 2.19, 875k¢ mean P&L) and the *gym-level* direction-feature agent (Sharpe −0.68, −13k¢ mean P&L) is precisely the gap our belief module aims to close. Simulation-level value agents run a full Kalman filter on every oracle observation; the gym agent sees only a single noisy tick. A belief tracker that approximates that Kalman filter is the natural bridge.

2 The Kalman Filter Belief Tracker

2.1 OU Process Model

The ABIDES oracle drives the fundamental according to a continuous-time OU process:

$$\mathrm{d}F_t = \kappa_{\text{oracle}}(\bar{r} - F_t)\,\mathrm{d}t + \sigma_{\text{fund}}\,F_t\,\mathrm{d}W_t.$$

Over a discrete timestep Δt (60 seconds, the agent wakeup interval), this discretises to the linear-Gaussian state-space model

$$F_{t+1} = \alpha F_t + (1-\alpha)\bar{r} + w_t, \qquad w_t \sim \mathcal{N}(0,\, Q), \tag{1}$$

$$y_t = F_t + v_t, \qquad v_t \sim \mathcal{N}(0,\, R), \tag{2}$$

where y_t is the observed mid-price, $\alpha = e^{-\kappa_{\text{oracle}}\,\Delta t}$ is the mean-reversion factor, Q is the per-step process noise variance, and R is the observation noise variance (bid-ask microstructure).

2.2 Parameter Derivation

Using the default RMSC04 calibration ($\bar{r} = 100{,}000$¢, $\kappa_{\text{oracle}} = 1.67 \times 10^{-16}$, $\sigma_{\text{fund}} = 5 \times 10^{-5}$, $\Delta t = 60$ s, observation noise fraction 0.5% of $\bar{r}$):

$$\begin{aligned}
\alpha &= e^{-\kappa_{\text{oracle}}\,\Delta t} \approx 1 - 1.0 \times 10^{-14} \approx 1,\\
Q &= \frac{(\bar{r}\,\sigma_{\text{fund}})^2}{2\kappa_{\text{oracle}}}\left(1-\alpha^2\right) \approx 1{,}496 \text{ cents}^2,\\
R &= (\bar{r} \times 0.005)^2 = 500^2 = 250{,}000 \text{ cents}^2.
\end{aligned}$$

Remark. Because $\kappa_{\text{oracle}} \cdot \Delta t \approx 10^{-14} \ll 1$, the mean-reversion term $\alpha \approx 1$ is negligible over a single 60-second step. The fundamental behaves as a near-random walk at intraday resolution. This makes the observation signal essential: without filtering, the agent has essentially no prior on where F_t is.

2.3 Kalman Predict-Update Equations

At each timestep the filter maintains a posterior mean μ_t and variance σ_t^2 over F_t, starting from $\mu_0 = \bar{r}$ and $\sigma_0^2 = Q/(1-\alpha^2)$ (stationary prior).

Predict step.

$$\begin{aligned}\hat{\mu}_t &= \alpha\,\mu_{t-1} + (1-\alpha)\,\bar{r},\\ \hat{\sigma}_t^2 &= \alpha^2\,\sigma_{t-1}^2 + Q.\end{aligned}$$

Update step (Kalman gain and posterior).

$$\begin{aligned}K_t &= \frac{\hat{\sigma}_t^2}{\hat{\sigma}_t^2 + R},\\ \mu_t &= \hat{\mu}_t + K_t\,(y_t - \hat{\mu}_t),\\ \sigma_t^2 &= (1-K_t)\,\hat{\sigma}_t^2.\end{aligned}$$

2.4 Steady-State Analysis

The posterior variance σ_t^2 converges to a fixed point P_∞ satisfying the discrete algebraic Riccati equation

$$P = \frac{Q + \sqrt{Q^2 + 4QR}}{2}.$$

With $Q = 1{,}496$ and $R = 250{,}000$:

$$P_\infty = 20{,}115 \text{ cents}^2, \qquad \sigma_\infty = \sqrt{P_\infty} \approx 142 \text{ cents}, \qquad K_\infty = \frac{P_\infty}{P_\infty + R} \approx 0.074.$$

The steady-state Kalman gain $K_\infty \approx 7.4\%$ means the filter trusts each new mid-price observation by 7.4% and holds 92.6% weight on its prior estimate. The posterior standard deviation of 142¢ represents the irreducible uncertainty about F_t given all past mid-price observations under the OU model.

2.5 Interpretation as EMA

Because $\alpha \approx 1$, the posterior mean reduces to an exponentially weighted moving average (EMA) of past mid-prices:

$$\mu_t \approx K_\infty\,y_t + K_\infty(1-K_\infty)\,y_{t-1} + K_\infty(1-K_\infty)^2\,y_{t-2} + \cdots$$

The effective half-life of this EMA is $\log(0.5)/\log(1-K_\infty) \approx 9$ timesteps (nine minutes at 60-second resolution). The belief feature $b_t = (\mu_t - \text{mid}_t)/\bar{r}$ is therefore the lagged EMA deviation from current price—a smoothed, noise-filtered version of the direction feature already in o_t.

2.6 Threshold Calibration

A critical implementation detail is setting the trading threshold τ such that b_t is non-trivially positive or negative with reasonable frequency. The correct calibration uses the steady-state posterior standard deviation:

$$\tau = \frac{\sigma_\infty}{2\,\bar{r}} = \frac{142}{2 \times 100{,}000} = 0.00071.$$

Setting $\tau = 0.002$ (approximately $1.4\,\sigma_\infty/\bar{r}$) caused the agent to hold every episode (zero P&L for all five test episodes) because b_t never exceeded the threshold. After reducing to $\tau = 0.0007$, the agent traded normally and achieved the results reported in Section 6.3. This calibration step is **required** whenever Q, R, or $\bar{r}$ change (e.g. across sweep parameter settings).

2.7 Implementation

The tracker is implemented in `belief_tracker.py` as two classes:

- `KalmanBeliefTracker`: stateless Kalman filter with `reset()` and `step(mid_price)` methods. Maintains (μ_t, σ_t^2) across timesteps.
- `BeliefAugmentedWrapper`: `gym.Wrapper` that reconstructs the mid-price from the info dict (`debug_mode=True` required) and appends two features to the observation:
  - $b_t^{\text{dev}} = (\mu_t - \text{mid}_t)/\bar{r}$
  - $b_t^{\text{std}} = \sigma_t/\bar{r}$

  New observation shape: $(9, 1)$ instead of $(7, 1)$.

3 The LSTM Belief Tracker

3.1 Motivation

The Kalman filter has two dependencies that limit its generality:

1. It requires knowledge of the OU parameters $(\kappa_{\text{oracle}}, \sigma_{\text{fund}}, \bar{r})$ at inference time.
2. It assumes the OU model is correct; deviations (e.g. momentum regimes, megashocks) are treated as additional observation noise.

The OAT sweep varied κ_{oracle} over $[8.35 \times 10^{-17}, 3.34 \times 10^{-16}]$ and σ_{fund} over $[1 \times 10^{-5}, 5 \times 10^{-4}]$, a $40\times$ range. A Kalman filter with fixed parameters will be suboptimal at the extremes of this range. An LSTM, having seen many episodes at different parameter settings, could in principle learn to *adapt* its belief update to the current regime.

3.2 Architecture

The LSTM belief network (`LSTMBeliefNet` in `lstm_belief_tracker.py`) maps a rolling window of market features to a predicted future return:

$$\begin{aligned} \text{Input}: &\ x_{t-T:t} \in \mathbb{R}^{T\times 4}, \quad T = 20 \text{ steps (20 minutes)}, \\ \text{Output}: &\ \hat{b}_t \in \mathbb{R}, \end{aligned}$$

where each feature vector is

$$x_s = \left[\Delta p_s/\bar{r},\ \text{imbalance}_s,\ \text{spread}_s/\bar{r},\ \text{direction}_s/\bar{r}\right].$$

All four inputs are normalised to be $O(1)$ independent of $\bar{r}$. The architecture is a single LSTM layer with hidden size 32 followed by a linear head:

$$h_T,\, c_T = \text{LSTM}(x_{t-T:t},\ h_0 = 0), \qquad \hat{b}_t = W\, h_T + \beta.$$

Total parameters: approximately $4 \times 32 \times 4 + 4 \times 32 \times 32 + 2 \times 32 + 32 = 4{,}704$.

3.3 Training Data Collection

Training data is generated by running 30 hold-only episodes (seeds 200–229, disjoint from test seeds 0–19) on the default RMSC04 calibration. Each episode has length 386 steps (full trading day at 60-second resolution). With a 3-step lookahead (see below), this yields approximately $(386 - 3) \times 30 \approx 11{,}490$ labelled samples.

3.4 Training Objective

The label for timestep t is the mean future mid-price return over the next $N = 3$ steps, normalised by $\bar{r}$:

$$y_t = \frac{1}{N\,\bar{r}} \sum_{k=1}^{N} (\mathrm{mid}_{t+k} - \mathrm{mid}_{t+k-1}).$$

Training minimises mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{t \in \mathcal{D}} \big(\hat{b}_t - y_t\big)^2.$$

Training used the Adam optimiser with learning rate 10^{-3}, batch size 256, and 30 epochs. Final training loss: $\mathcal{L} \approx 10^{-8}$.

3.5 Inference Wrapper

The `LSTMBeliefWrapper` maintains a rolling buffer of the last $T = 20$ feature vectors. At each step it pads the buffer (zero-padding for early steps) and passes it through the frozen model to produce $\hat{b}_t$. The wrapper appends two features to the observation matching the Kalman format: $\mathrm{obs}[-2] = \hat{b}_t$ (predicted future return), $\mathrm{obs}[-1] = 0$ (no uncertainty estimate from LSTM).

4 The Particle Filter Belief Tracker

4.1 Motivation

The Kalman filter is the optimal estimator for linear-Gaussian models, but the ABIDES microstructure introduces two deviations from linearity that Kalman treats poorly:

- **Megashocks.** RMSC04 has a small per-step probability ($\lambda_a \approx 2.78 \times 10^{-18}$) of a large signed impulse to F_t (mean 1,000¢, std $\approx$ 224¢). A Kalman filter treats this as a large innovation and absorbs it slowly; a particle cloud can spread to cover both pre- and post-shock hypotheses simultaneously.
- **Non-Gaussian microstructure noise.** Bid-ask bounce and inventory effects create fat-tailed, multi-modal observation noise that a Gaussian R can only approximate.

The bootstrap (SIR) particle filter drops both assumptions at the cost of $O(N)$ particles.

4.2 Algorithm

The filter maintains N weighted particles $\{F_t^{(i)}, w_t^{(i)}\}_{i=1}^{N}$ that collectively approximate the posterior $p(F_t \mid y_{1:t})$. Each timestep proceeds in four stages:

1. **Systematic resample.** Draw N new particles from the current weighted distribution using systematic (Carpenter et al. 1999) resampling, which has $O(N)$ cost and lower variance than multinomial sampling.

2. **Propagate.** Each resampled particle evolves under the discretised OU dynamics (identical to the Kalman state equation, Eq. 1). Optional megashocks are applied independently to each particle with probability p_{shock} per step.

3. **Weight.** Assign importance weights $w^{(i)} \propto p(y_t \mid F_t^{(i)}) = \mathcal{N}(y_t;\, F_t^{(i)},\, R)$.

4. **Posterior statistics.** Compute the weighted mean and std:

$$\mu_t = \sum_i w_t^{(i)}\, F_t^{(i)}, \qquad \sigma_t = \sqrt{\sum_i w_t^{(i)}\, (F_t^{(i)} - \mu_t)^2}.$$

Weight degeneracy (all weight on one particle) is detected by the *effective sample size* $\text{ESS} = 1/\sum_i (w_t^{(i)})^2 \in [1, N]$. When the filter detects catastrophic weight collapse ($\sum w < 10^{-300}$), it re-initialises from the prior rather than propagating a degenerate cloud.

4.3 Implementation

`particle_filter_tracker.py` implements two classes:

- `ParticleFilterTracker`: stateless SIR filter with `reset()` and `step(mid_price)` methods. Default: $N = 300$ particles, initialised from $\mathcal{N}(\bar{r},\, R_{\text{std}})$ rather than the OU stationary prior (whose variance $\approx 2.7 \times 10^8$ causes immediate weight degeneracy for near-random-walk kappa).

- `ParticleBeliefWrapper`: drop-in replacement for `BeliefAugmentedWrapper` with identical obs-contract ($\text{obs}[-2] = b_t^{\text{dev}}$, $\text{obs}[-1] = \sigma_t/\bar{r}$).

5 The Kim Regime-Switching Filter

5.1 Motivation

The Kalman filter assumes a single fixed market dynamic. RMSC04 populates the market with three structurally different agent types whose aggregate influence can shift across episodes: value agents drive mean-reversion, momentum agents drive trending, and noise agents dominate during low-information periods. A tracker that explicitly models regime uncertainty can hedge its belief update against these different dynamics and report regime probabilities as additional policy features.

5.2 Regimes

We define $K = 3$ regimes with distinct OU parameters:

Regime	α	Q multiplier	R multiplier
VALUE (0)	α_{OU} (mean-reverts)	$1\times$	$1\times$
MOMENTUM (1)	1.0 (random walk)	$20\times$	$2\times$
NOISE (2)	α_{OU}	$0.3\times$	$15\times$

Table 2: Per-regime dynamics for the Kim filter. MOMENTUM uses $\alpha = 1$ (no mean-reversion) and large Q (fast-drifting fundamental). NOISE uses small Q and very large R (uninformative observations).

5.3 Algorithm (Kim 1994 Collapse Approximation)

Each step maintains a regime probability vector $\pi_t \in \Delta^{K-1}$ and per-regime posterior Gaussians $(\mu_t^j, \sigma_t^{2,j})$ for $j \in \{0, 1, 2\}$.

1. **Predict.** For every pair (i, j) of prior regime i and current regime j, run a Kalman predict step using regime-j dynamics starting from the regime-i posterior. Compute the Gaussian log-likelihood $\log \ell_{ij}$ of the mid-price observation under the predicted distribution.

2. **Joint update.** Form the joint weight $\log p(S_{t-1} = i,\, S_t = j \mid y_{1:t}) \propto \log \pi_{t-1}^i + \log \Pi_{ij} + \log \ell_{ij}$, where Π is the $K \times K$ regime transition matrix with diagonal $\Pi_{jj} = 0.97$ (regimes are persistent).

3. **Marginalise.** $\pi_t^j = \sum_i p(S_{t-1} = i,\, S_t = j \mid y_{1:t})$.

4. **Kim collapse.** Merge the K^2 posteriors back into K Gaussians by computing the conditional mixture mean and variance, preserving the first two moments of each per-current-regime mixture.

The output is a mixture mean $\mu_t^{\text{mix}} = \sum_j \pi_t^j \mu_t^j$, mixture std σ_t^{mix}, and regime probability vector π_t.

5.4 Regime-Aware Trading Rule

The Kim wrapper appends five features to the observation: $[b_t^{\text{dev}},\, p_{\text{value}},\, p_{\text{mom}},\, p_{\text{noise}},\, b_t^{\text{std}}]$. The regime-aware agent blends two complementary signals:

$$\text{signal}_t = p_{\text{value}} \cdot b_t^{\text{dev}} + p_{\text{momentum}} \cdot \frac{\Delta p_{t-1}}{\bar{r}},$$

where the value signal is contrarian (buy when fundamental is above market) and the momentum signal is trend-following. The noise regime probability suppresses both signals by diluting the weights.

6 Experiments

6.1 Setup

All experiments use the `markets-daily_investor-v0` gym environment with `debug_mode=True` (required to recover mid-price from the info dict) and the following common configuration:

Parameter	Value
`background_config`	`rmsc04`
`timestep_duration`	`60s`
`starting_cash`	1,000,000¢
`order_fixed_size`	10 shares
`state_history_length`	4
`r_bar`	100,000¢
`kappa_oracle`	1.67×10^{-16}
`fund_vol`	5×10^{-5}
Test seeds	0–19 (20 episodes)
LSTM training seeds	200–229 (30 episodes, hold policy)

Table 3: Common experimental configuration.

6.2 Agents

VALUE (baseline). Uses the raw direction feature $o_t[3] = \text{mid}_t - \text{last_transaction}_t$ with threshold ± 0.75. This is the best-performing hand-coded gym strategy from the sweep.

$$a_t = \begin{cases} \text{BUY} & \text{if } o_t[3] < -0.75, \\ \text{SELL} & \text{if } o_t[3] > +0.75, \\ \text{HOLD} & \text{otherwise.} \end{cases}$$

BELIEF_KALMAN. Uses the Kalman posterior deviation $b_t^{\text{dev}} = (\mu_t - \text{mid}_t)/\bar{r}$ from `BeliefAugmentedWrapper` with threshold $\tau = 0.0007$ (half the steady-state posterior std):

$$a_t = \begin{cases} \text{BUY} & \text{if } b_t^{\text{dev}} > +0.0007 \ \ (\mu_t > \text{mid}_t : \text{undervalued}), \\ \text{SELL} & \text{if } b_t^{\text{dev}} < -0.0007, \\ \text{HOLD} & \text{otherwise.} \end{cases}$$

BELIEF_PARTICLE. Uses the particle-filter posterior mean μ_t^{PF} from `ParticleBeliefWrapper` ($N = 300$) with the same threshold $\tau = 0.0007$ as Kalman:

$$a_t = \begin{cases} \text{BUY} & \text{if } (\mu_t^{\text{PF}} - \text{mid}_t)/\bar{r} > +0.0007, \\ \text{SELL} & \text{if } (\mu_t^{\text{PF}} - \text{mid}_t)/\bar{r} < -0.0007, \\ \text{HOLD} & \text{otherwise.} \end{cases}$$

BELIEF_KIM. Uses the blended regime signal from `KimBeliefWrapper` with the same value-signal threshold $\tau = 0.0007$ and momentum threshold 0.0001:

$$a_t = \begin{cases} \text{BUY} & \text{if signal}_t > +0.0007, \\ \text{SELL} & \text{if signal}_t < -0.0007, \\ \text{HOLD} & \text{otherwise.} \end{cases}$$

BELIEF_LSTM. Uses the LSTM-predicted future return $\hat{b}_t$ from `LSTMBeliefWrapper` with threshold $\tau = 0$ (pure sign of prediction):

$$a_t = \begin{cases} \text{BUY} & \text{if } \hat{b}_t > 0, \\ \text{SELL} & \text{if } \hat{b}_t < 0, \\ \text{HOLD} & \text{never (threshold is zero).} \end{cases}$$

6.3 Main Results

Agent	Mean P&L (¢)	Std (¢)	Sharpe	Win%	Mean DD
VALUE	−13,832	20,284	−0.682	20%	0.0288
BELIEF_KALMAN	+7,670	15,359	+0.499	60%	0.0227
BELIEF_PARTICLE	+8,703	20,483	+0.425	65%	0.0259
BELIEF_KIM	+6,224	19,565	+0.318	55%	0.0210
BELIEF_LSTM	−48,500	342,703	−0.142	40%	0.351

Table 4: Five-way comparison over 20 test episodes (seeds 0–19). LSTM was trained on hold-only seeds 200–229 (30 episodes). Ranked by Sharpe: Kalman > Particle > Kim > LSTM > VALUE. All three model-based belief trackers beat the raw direction-feature baseline.

The three model-based trackers all outperform the VALUE baseline:

- **BELIEF_KALMAN**: best Sharpe (+0.499), mean P&L +7,670¢, win rate 60%, lowest variance of the group (15,359¢ std).
- **BELIEF_PARTICLE**: highest mean P&L (+8,703¢) and win rate (65%), Sharpe +0.425. Higher std than Kalman (20,483¢) because stochastic resampling adds sampling noise.
- **BELIEF_KIM**: lowest mean drawdown (0.021), Sharpe +0.318. More conservative than Kalman/Particle: the regime-blended signal is attenuated by the noise-regime probability, reducing trade frequency.
- **BELIEF_LSTM**: worst by mean P&L (−48,500¢) and by far the most risky (drawdown 0.35, fourteen times higher than Kalman).

7 Analysis

7.1 Why the Kalman Filter Works

The Kalman filter succeeds because it correctly models the data-generating process. With $K_\infty = 0.074$, the posterior mean μ_t is a 9-step EMA of mid-prices: it is a smoothed representation of recent price history that changes slowly and oscillates around the true fundamental. Crucially, because μ_t integrates evidence over the full episode, it does not overreact to single-tick noise.

The resulting belief deviation $b_t^{\text{dev}} = (\mu_t - \text{mid}_t)/\bar{r}$ oscillates symmetrically around zero: when the price rises above the EMA, $b_t^{\text{dev}} < 0$ (sell signal); when the price falls below, $b_t^{\text{dev}} > 0$ (buy signal). This natural oscillation prevents the agent from accumulating a persistent one-sided inventory, keeping risk controlled (max drawdown 0.023).

7.2 Why the Particle Filter Nearly Matches Kalman

Under the default RMSC04 calibration the OU model is linear-Gaussian and megashocks are extremely rare ($\lambda_a \cdot \Delta t \approx 10^{-10}$ per 60-second step). In this regime the particle filter with $N = 300$ particles converges to the Kalman solution in the large-N limit; both trackers estimate approximately the same posterior mean. The particle filter's slight Sharpe disadvantage (+0.425 vs +0.499) is attributable purely to Monte Carlo sampling noise: with finite N, the weighted mean μ_t^{PF} fluctuates more than the analytic Kalman mean, occasionally generating spurious trades. Conversely, the particle filter achieves a higher mean P&L (+8,703 vs +7,670¢) and win rate (65% vs 60%), suggesting that in the episodes where both agents trade, the particle filter makes slightly better decisions—consistent with better handling of the few genuine tail events in the simulation. The particle filter's advantage would be more pronounced with megashocks enabled ($\lambda_a \gg 0$), where the Kalman filter is known to recover slowly.

7.3 Why the Kim Filter Is More Conservative

The Kim filter introduces an additional source of uncertainty: the regime posterior. Under the default RMSC04 parameters, the market does not exhibit strong regime separation—the VALUE/MOMENTUM/NOI dynamic parameters are calibrated from the true generator, but at any given timestep the filter spreads probability across all three regimes rather than collapsing to one. The blended signal $\mathrm{signal}_t = p_{\mathrm{value}} \cdot b_t^{\mathrm{dev}} + p_{\mathrm{momentum}} \cdot \Delta p_{t-1}/\bar{r}$ is thus attenuated by the noise-regime probability weight p_{noise}, which dilutes both value and momentum contributions. This makes the effective threshold looser in practice (fewer trades), reducing both upside and downside exposure. The lowest mean drawdown (0.021) confirms this: the Kim filter effectively self-regulates position-taking when regime uncertainty is high. The regime probability features $[p_{\mathrm{value}}, p_{\mathrm{momentum}}, p_{\mathrm{noise}}]$ are the most distinctive output of this tracker and are expected to add value as *input features* to an RL policy even if the standalone Kim threshold rule underperforms Kalman.

7.4 Threshold Calibration and the All-Hold Failure Mode

A non-obvious implementation failure occurred during initial testing: setting $\tau = 0.002$ caused the Kalman agent to hold every episode with exactly zero P&L across all five test episodes. The diagnostic is as follows.

The steady-state posterior standard deviation is $\sigma_\infty = 142$¢, corresponding to $b_t^{\mathrm{dev}} \sim O(0.00142)$. A threshold of $0.002 \approx 1.4\,\sigma_\infty/\bar{r}$ is larger than the typical signal magnitude, so the condition $|b_t^{\mathrm{dev}}| > \tau$ is almost never satisfied. This is analogous to setting a detection threshold above the noise floor in signal processing.

The fix was to set $\tau = \sigma_\infty/(2\,\bar{r}) = 0.0007$, which the signal exceeds roughly half the time at steady state. This threshold is fully determined by the Riccati solution and should be recomputed whenever κ_{oracle}, σ_{fund}, or the observation noise fraction changes.

7.5 Why the LSTM Fails

The LSTM failure has a clear mechanistic explanation.

Near-zero predictions. The training target $y_t = \Delta\bar{\mathrm{m}}\mathrm{id}_3/\bar{r}$ is numerically tiny (typical magnitude ≈ 0.0004) because the fundamental is a near-random walk over 3 steps. The MSE loss converged to $\approx 10^{-8}$, which means the model has learned to predict values very close to zero—the optimal constant prediction under MSE for an approximately zero-mean target.

Unbounded position accumulation. With threshold $\tau = 0$, *any* non-zero prediction fires a trade. If $\hat{b}_t$ is systematically slightly positive (or negative) over consecutive steps, the agent accumulates an ever-larger position without a reversal signal. At 10 shares per order and 386 steps, a consistently positive prediction could build a long position of up to 3,860 shares, with P&L scaling linearly (or super-linearly) with the price drift over that period. This produces the observed fat-tailed P&L distribution (std = 342,703¢ versus 15,359¢ for Kalman).

Root cause. The combination of (a) tiny-magnitude predictions, (b) zero threshold, and (c) no position cap creates a high-variance lottery rather than a belief tracker. This is not a failure of the LSTM architecture per se—it is a calibration and training-signal failure.

7.6 How to Fix the LSTM

Three targeted changes would make the LSTM competitive with Kalman:

1. **Use the Kalman output as the training label** (instead of raw future returns). Kalman labels have magnitude $O(0.001)$, a 2.5× larger signal; the LSTM learns to reproduce a properly-calibrated deviation signal rather than a near-zero noise floor. This also allows direct comparison: does the LSTM match Kalman on the same parameters? Does it outperform on out-of-distribution parameters?

2. **Calibrate the trading threshold** using the same percentile analysis as Kalman. Collect LSTM outputs on held-out episodes, compute the 50th-percentile absolute value, and use that as τ.

3. **Add a soft position limit** (e.g. refuse BUY if holdings > 50, refuse SELL if holdings < -50) to prevent runaway inventory regardless of tracker quality.

8 Comparison of All Trackers

Property	Kalman	Particle (SIR)	Kim (RS-Kalman)	LSTM
Training?	No. Zero-shot.	No. Zero-shot.	No. Zero-shot.	Yes. ∼30 rollouts.
Model	Linear-Gaussian OU (exact).	Non-parametric; handles fat tails and megashocks.	Mixture of 3 OU regimes.	No model; data-driven.
Interpretability	μ_t, σ_t semantically meaningful.	μ_t, σ_t from weighted cloud.	μ_t, σ_t, *and* π_t (regime probs).	Black-box hidden state.
Extra obs features	2: $b^{\text{dev}}, b^{\text{std}}$.	2: same.	5: $b^{\text{dev}}, p_V, p_M, p_N, b^{\text{std}}$.	2: $\hat{b}, 0$.
Parameter sensitivity	Degrades when κ or σ_{fund} drift.	Same (uses same OU params).	Partly robust: regime probs adapt if one regime fits better.	Robust if trained on diverse settings.
Complexity / cost	$O(1)$ per step.	$O(N)$ per step (N=300: ∼10× slower than Kalman).	$O(K^2)$ per step (K=3: small).	$O(T \cdot h)$ per step; requires pretraining.
Performance (20 eps)	Sharpe +0.50, PnL +7,670¢, win 60%.	Sharpe +0.43, PnL +8,703¢, win 65%.	Sharpe +0.32, PnL +6,224¢, win 55%.	Sharpe −0.14, PnL −48,500¢, win 40%.
Primary failure mode	Miscalibrated τ → all-hold.	Finite-N noise → spurious trades.	Regime uncertainty attenuates signal → fewer trades.	Near-zero predictions + zero threshold → unbounded inventory.

Table 5: Qualitative and quantitative comparison of the four belief trackers. All three model-based trackers beat the VALUE baseline (Sharpe −0.68).

9 Next Steps

9.1 Immediate: Fix the LSTM Tracker

Apply the three fixes from Section 7.5 and re-run `belief_comparison.py --lstm-model <path>`:

1. Use the Kalman posterior mean as the training label (magnitude $O(0.001)$ vs near-zero raw returns).
2. Calibrate τ from the 50th-percentile absolute LSTM output on hold-out episodes.
3. Add a soft position cap (e.g. $|\text{holdings}| \leq 50$) to prevent runaway inventory.

With these fixes the LSTM should at least match Kalman on default parameters and potentially outperform on out-of-distribution sweep settings.

9.2 RL Integration

All three working belief trackers are ready to be plugged into the PPO training pipeline described in v3.tex. The recommended integration priority:

1. **Kalman features (Run 5 equivalent).** Wrap the environment with `BeliefAugmentedWrapper` before the Gymnasium adapter in `train_ppo_daily_investor.py`. Observation shape: $(7, 1) \to (9, 1)$ (two new features). No reward changes needed. Train PPO and compare against the unaugmented baseline (Run 1).
2. **Kim regime probabilities.** Wrap with `KimBeliefWrapper`: observation shape $(7, 1) \to (12, 1)$ (five new features: $b^{\text{dev}}, p_V, p_M, p_N, b^{\text{std}}$). The regime probabilities are the most distinctive

feature of this tracker and are expected to add policy-conditioning value that neither Kalman nor Particle can provide.

3. **Oracle regime label (Run 2 from v3.tex).** Since ABIDES is a simulator, the ground-truth regime is known at episode start. Pass a one-hot $[p_V, p_M, p_N]$ derived from background agent counts as an oracle belief. If oracle outperforms the Kim filter, the Kim filter's regime inference is the bottleneck; if they are comparable, the filter is tracking well.

4. **Multi-regime training.** Randomise κ_{oracle}, σ_{fund}, and agent counts across episodes during training. The Kim filter and LSTM adapt; Kalman degrades gracefully. This is the key experiment to determine which tracker wins *out-of-distribution*.

9.3 Particle Filter with Megashocks Enabled

The current comparison uses the RMSC04 default $\lambda_a = 2.78 \times 10^{-18}$ (negligible per-step shock probability). Enabling megashocks ($\lambda_a = 10^{-6}$, say) should reveal the particle filter's advantage: the Gaussian Kalman filter cannot maintain a bimodal posterior after a shock, while the particle cloud can. Re-run `belief_comparison.py` with `megashock_lambda_a` elevated in the background config and compare Kalman vs. Particle (N=300 vs. N=1000 for accuracy) on 50 episodes.

9.4 Generalisation Across Sweep Parameters

The OAT sweep showed performance sensitivity to σ_{fund} and κ_{oracle}. Re-running `belief_comparison.py` at sweep extremes ($\sigma_{\text{fund}} \in \{10^{-5}, 5 \times 10^{-4}\}$, $\kappa_{\text{oracle}} \in \{8.35 \times 10^{-17}, 3.34 \times 10^{-16}\}$) will quantify:

- Kalman sensitivity to model mismatch (known to degrade).
- Particle filter sensitivity (same OU params $\rightarrow$ same degradation).
- Kim filter robustness (regime posteriors may compensate for parameter drift if the active regime shifts).
- LSTM robustness if retrained on diverse settings.